

Intelligenza Artificiale

Michele Castenedoli

1 ottobre 2026

Indice

1	Introduzione all’Intelligenza Artificiale	2
1.1	Definizione e Storia	2
1.2	Tipi di Intelligenza Artificiale	2
2	Agenti Intelligenti	7
2.1	Definizione	7
2.2	Razionalità	9
2.3	PEAS	11
2.4	Tipi di Ambiente	12
2.5	Tipi di Problemi	13
2.6	Programma Agente	14
3	Risolvere Problemi con la Ricerca	19
3.1	Problem Solving Agents	19
3.2	Come Formulare un Problema	20
3.3	Tree-Search Algorithm	22

Capitolo 1

Introduzione all'Intelligenza Artificiale

1.1 Definizione e Storia

Nel 1950 Alan Turing pubblica un articolo intitolato “Computing Machinery and Intelligence” in cui viene proposto un esperimento per determinare se una macchina può essere considerata dotata di intelligenza. Tale esperimento, noto come *Test di Turing*, coinvolge un interrogatore umano che comunica con due entità nascoste: una **macchina** e un **essere umano**. L'interrogatore sottopone domande ad entrambe le entità e, basandosi sulle risposte fornite, deve determinare quale delle due è la macchina. Se l'interrogatore non è in grado di distinguere le due parti, la macchina è considerata intelligente.

In seguito l'attenzione si è spostata sulla ricerca di metodi utili per risolvere problemi che richiedono l'intelligenza umana mediante l'utilizzo di algoritmi e modelli matematici fino ad arrivare all'introduzione di reti neurali e deep learning.

Intelligenza Artificiale

L'**Intelligenza Artificiale** è la disciplina che si occupa di studiare come **simulare** l'intelligenza umana in scenari complessi.

1.2 Tipi di Intelligenza Artificiale

1.2.1 Autonomous Agents

Gli **Autonomous Agents** sono sistemi che percepiscono l'ambiente e agiscono autonomamente per raggiungere determinati obiettivi.

1.2.2 Data Analysis

Consiste nell'utilizzo di algoritmi per analizzare grandi quantità di dati ed estrarre informazioni utili e possibili correlazioni complesse.

1.2.3 Machine Learning

È lo sviluppo di algoritmi che permettono a modelli di apprendere da *dati di esempio* e migliorare le loro prestazioni nel tempo senza essere esplicitamente programmati: a differenza della programmazione manuale e tradizionale, dove lo sviluppatore scrive una serie di istruzioni codificate in un linguaggio di programmazione, si fornisce soltanto un algoritmo di base e una grande quantità di dati da cui apprendere. Il sistema analizza tali dati per trovare possibili pattern, correlazioni e regole statistiche necessarie per svolgere il compito.

L'apprendimento automatico è suddiviso in tre categorie principali:

- **Unsupervised Learning:** il modello viene addestrato nel corso del tempo su un insieme di dati non etichettati (senza alcun “tag”, categoria, o classificazione assegnati a priori da un essere umano). Il target è scoprire diverse strutture nascoste o pattern nei dati senza l'utilizzo di risposte corrette predefinite.
- **Supervised Learning:** il modello viene addestrato su un insieme di dati etichettati, dove ogni esempio di input è associato ad una risposta corretta. L'obiettivo è che il modello impari a mappare gli input alle risposte corrette.
- **Reinforcement Learning:** il modello impara attraverso continue interazioni con l'ambiente, ricevendo ricompense o penalità a seconda delle azioni intraprese. L'obiettivo è massimizzare la ricompensa totale nel tempo. Pertanto il sistema valuta le conseguenze delle scelte a lungo termine anziché cercare un vantaggio immediato.

Esempio: il sistema che simula un giocatore di scacchi potrebbe sacrificare la regina se prevede che in due mosse sarà in grado di dare scacco matto, raggiungendo così la massima ricompensa disponibile.

1.2.4 Time Series Analysis

L'analisi delle serie temporali è un'area dell'apprendimento automatico che si concentra sull'analisi di dati collezionati nel tempo. Le serie temporali sono sequenze di dati misurati ad intervalli regolari (temperatura giornaliera,

prezzi delle azioni o dati di vendita mensili). In questo caso, l'obiettivo è identificare pattern, tendenze, stagionalità¹ per fare previsioni future.

Gli approcci comuni per l'analisi delle serie temporali sono:

- **Riconoscimento di anomalie e cause:** processo di identificazione di dati o eventi che si discostano significativamente dal comportamento normale o atteso. Queste anomalie possono fornire indicatori di problemi, errori o situazioni non previste che richiedono attenzione.
- **Generative Transformers:** sono una classe di modelli che permettono di predire il prossimo elemento di una sequenza di dati partendo dagli elementi precedenti (token di un testo, pixel di un'immagine). Si introduce il concetto di **attenzione**: meccanismo usato per pesare l'importanza delle diverse parti della sequenza in input durante la generazione di output.

Esempio: Se il modello di apprendimento sta elaborando la sequenza di token relativa alla frase "ho mangiato una pesca", il meccanismo alla base di attenzione assegnerà un peso (punteggio) molto elevato alle parole "mangiato" e "una" per aiutare il sistema a capire che ci si sta riferendo al frutto e non all'hobby di pescare.

1.2.5 Intelligent Agents

Un agente intelligente è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce sull'ambiente per raggiungere un obiettivo specifico e svolgere un determinato compito. Gli elementi cardine di un agente intelligente includono:

- **Performance Measure:** misurazione del successo (efficacia) dell'agente nel raggiungere i suoi obiettivi.
- **Rationality:** l'agente deve agire in modo da massimizzare la sua performance measure attesa, dati la sequenza percettiva e la conoscenza a priori di cui dispone.

¹Stagionalità: la presenza di fluttuazioni, variazioni o pattern regolari prevedibili che si ripetono a intervalli regolari di tempo.

Differenza – Intelligent Agents vs Autonomous Agents

I due concetti si sovrappongono in gran parte, ma pongono l'accento su aspetti diversi. L'**intelligenza** riguarda la *qualità delle decisioni*: un agente è intelligente (razionale) se sceglie le azioni che massimizzano la performance measure attesa. L'**autonomia** riguarda invece *quanto l'agente dipende dal progettista*: un agente è autonomo nella misura in cui il suo comportamento si basa sulla propria esperienza (percezioni e apprendimento) anziché solo sulla conoscenza inserita a priori, e opera senza un intervento umano continuo. Le due proprietà sono quindi indipendenti: un termostato con regole fisse è un agente ma non è autonomo, mentre un'auto a guida autonoma che impara e si adatta a situazioni nuove è sia intelligente sia autonoma.

1.2.6 Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali in ambienti stocastici. Gli elementi principali sono:

- **States**: insieme S degli stati; uno stato $s \in S$ rappresenta l'ambiente in un dato momento
- **Actions**: insieme A delle azioni che l'agente può intraprendere
- **Transition model**: effetto che le azioni hanno sull'ambiente. Poiché gli effetti potrebbero essere incerti, il modello assegna una probabilità a ogni possibile stato successivo

$$T(s, a, s') = P(s' \mid s, a)$$

Dove T è la funzione di transizione e esprime la probabilità di capitare nello stato s' sapendo che si è nello stato s e che l'azione a viene eseguita.

Approfondimento – Transition Model

Da notare che T è solo un altro modo per descrivere la probabilità condizionata di capitare in s' sapendo che si parte dallo stato s e si esegue l'azione a .

- **Reward**: valore **immediato** dell'esecuzione di un'azione

$$R : S \times A \times S \rightarrow \mathbb{R}$$

Si definisce la ricompensa che l'agente guadagna raggiungendo quel passo:

- Il primo S indica lo stato di partenza
- A è l'azione che è stata eseguita
- Il secondo S indica lo stato in cui si è giunti
- **Policy:** strategia che l'agente utilizza per decidere quale azione intraprendere in ogni stato con l'obiettivo di massimizzare la ricompensa totale attesa nel tempo

$$\pi : S \rightarrow A$$

Alla base del modello c'è la **proprietà di Markov**, da cui prende il nome: la probabilità di raggiungere lo stato successivo dipende *soltanto* dallo stato corrente e dall'azione scelta, e non dalla storia degli stati e delle azioni precedenti:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

In altre parole, lo stato corrente riassume tutta l'informazione rilevante per decidere come agire.

Capitolo 2

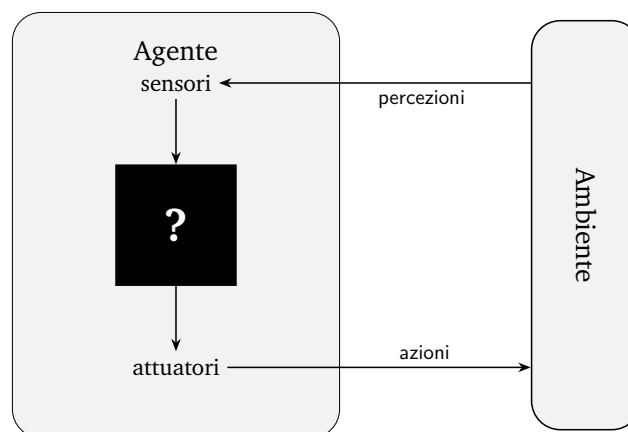
Agenti Intelligenti

2.1 Definizione

Agente Intelligente

Un **agente** è qualsiasi cosa possa essere vista come un sistema che percepisce il suo ambiente attraverso dei **sensori** e agisce su di esso tramite **attuatori**.

Esempio: L'agente umano possiede sensori come occhi e orecchie, mentre può usare come attuatori mani, gambe, tratto vocale. Un agente robotico potrebbe invece avere telecamere e telemetri a infrarossi per sensori e diversi motori come attuatori.



Si usa il termine **percezione** per indicare i dati che i sensori raccolgono. La **Sequenza Percettiva** di un agente è la storia di tutto ciò che l'agente

ha raccolto in tutta la sua esistenza. In generale, la scelta dell'azione di un agente in un qualsiasi istante può dipendere dalla conoscenza integrata in esso e dall'intera sequenza percettiva osservata fino a quel momento, ma non da qualcosa che l'agente non abbia percepito.

Specificando l'azione che un agente può scegliere in qualunque istante, si definisce l'agente in modo completo. In termini matematici, il comportamento di un agente è descritto dalla funzione agente, che descrive la corrispondenza tra una qualsiasi sequenza percettiva e una specifica azione.

La funzione agente assume la seguente forma:

$$f : \mathcal{P}^* \rightarrow \mathcal{A}$$

In particolare:

- $\mathcal{P}$ indica l'insieme delle singole percezioni
- $*$ (*chiusura di Kleene*) è l'operatore che, applicato a un insieme, restituisce l'insieme di *tutte* le sequenze finite (compresa quella vuota) formate dai suoi elementi
- $\mathcal{P}^*$ è quindi l'insieme di tutte le possibili sequenze percettive: ogni suo elemento rappresenta una possibile cronologia delle percezioni registrate fino a un certo momento
- $\mathcal{A}$ indica l'insieme delle possibili azioni

Internamente, la funzione agente è implementata da un **programma agente**.

Differenza – Programma Agente vs Funzione Agente

Mentre la funzione agente è una descrizione matematica astratta, il programma agente è una sua implementazione fisica concreta, in esecuzione all'interno del sistema fisico.

Esempio: Un mondo dell'aspirapolvere con due sole posizioni. Ogni posizione può essere pulita o sporca, l'agente può andare a sinistra o a destra e può pulire il riquadro in cui si trova.

Sequenza Percettiva	Azione
[A, Pulito]	Destra
[A, Sporco]	Aspira
[B, Pulito]	Sinistra
[B, Sporco]	Aspira
[A, Pulito], [A, Pulito]	Destra
[A, Pulito], [A, Sporco]	Aspira
$\vdots$	$\vdots$

Se un agente ha $|\mathcal{P}|$ possibili percezioni, dopo T passi temporali, la funzione agente ha $\sum_{t=1}^T |\mathcal{P}|^t$ voci. Se lo storico di percezioni è irrilevante, cioè se l'azione dipende soltanto dalla percezione corrente, l'agente viene detto **Simple Reflex Agent**.

L'obiettivo è progettare piccoli programmi agente per rappresentare enormi funzioni agente.

Un possibile programma agente

```
function AGENTE-REATTIVO-ASPIRAPOLVERE([posizione, stato]) returns azione
  if stato = Sporco then return Aspira
  else if posizione = A then return Destra
  else if posizione = B then return Sinistra
```

Questo programma è adeguato se la performance measure prende in considerazione solo il numero di stanze pulite. Se subentrassero altre metriche che portano ad una definizione della performance measure diversa (stato della batteria, percentuale di ricarica, minimizzare il numero di spostamenti), tale programma non è in grado di esprimere la cosa migliore da fare. Da notare che l'agente che implementa tale programma è un **Simple Reflex Agent**: l'azione dipende soltanto dalla percezione corrente.

2.2 Razionalità

2.2.1 Comportarsi Correttamente: il concetto di razionalità

Definizione

Un **agente razionale** è un agente che fa la cosa giusta.

Esistono diverse accezioni dell'espressione "cosa giusta", ma l'IA rimane fedele alla nozione di **conseguenzialismo**: si valuta il comportamento di un agente considerando le conseguenze. Quando un agente viene inserito in un ambiente, genera una sequenza di azioni in base alle percezioni che riceve. Questa sequenza di azioni porta l'ambiente ad attraversare una sequenza di stati: se tale sequenza è desiderabile, significa che l'agente si è comportato bene. Questa nozione di desiderabilità è catturata da una misura di prestazione che valuta la sequenza di stati dell'ambiente.

In un dato momento, ciò che è razionale dipende da quattro fattori:

- la misura di prestazione che definisce il criterio del successo
- la conoscenza pregressa dell'ambiente da parte dell'agente

- le azioni che l'agente può eseguire
- la sequenza percettiva dell'agente fino allo stato corrente

Tutto ciò porta a una definizione molto formale di **agente razionale**:

Definizione

Per ogni possibile sequenza di percezioni, un agente razionale dovrebbe scegliere un'azione che massimizzi il valore atteso della sua misura di prestazione, date le informazioni fornite dalla sequenza percettiva e da ogni ulteriore conoscenza dell'agente.

La razionalità allora non è altro che un problema di ottimizzazione sulla performance measure: un agente razionale sceglie l'azione che massimizza il valore atteso della misura di prestazione, data la sequenza percettiva accumulata fino a quel momento.

Notare come essere razionale non vuol dire essere onnisciente.¹ Infatti le percezioni potrebbero non fornire tutte le informazioni rilevanti in un dato istante. Pertanto, i risultati potrebbero essere diversi da quelli previsti.

Differenza – Razionalità vs Perfezione

La razionalità massimizza la prestazione **attesa**, mentre la perfezione massimizzerebbe la prestazione **effettiva**. Un agente razionale non può essere giudicato per esiti che non poteva prevedere: se attraverso la strada dopo aver controllato che sia libera e vengo colpito da un oggetto caduto dall'alto, la mia scelta è stata comunque razionale.

2.2.2 Esercizio

L'esercizio propone di analizzare un ambiente composto da tre stanze (A, B, C) e due robot (r_1, r_2). Il robot r_1 può pattugliare le stanze A e B partendo da A, mentre r_2 può pattugliare B e C partendo da C. Assumendo un tempo di spostamento nullo, l'obiettivo è definire un comportamento razionale capace di minimizzare l'inattività media (*average idleness*), definita come la frazione del tempo in cui una stanza non è visitata da alcun robot.

Per affrontare il problema, possiamo analizzare tre diverse strategie:

- **Prima strategia (Vettore di inattività $[0, 1, 0]$):** I robot non cambiano mai la loro posizione iniziale. Il robot r_1 rimane in A e r_2 in C. Di

¹Possedere conoscenza illimitata su qualsiasi cosa.

conseguenza, le stanze A e C hanno inattività nulla, ma la stanza B non viene mai coperta, generando un'inattività pari a 1.

- **Seconda strategia (Vettore di inattività $[1/2, 1/2, 0]$):** Si decide di muovere soltanto il primo robot (r_1), che si sposterà continuamente da A a B e viceversa, mentre r_2 resta fermo in C. In questo scenario, le stanze A e B rimangono scoperte per metà del tempo.
- **Terza strategia (Vettore di inattività $[1/4, 1/2, 1/4]$):** Questa strategia consente di far giocare un ruolo attivo anche al secondo robot (r_2), coordinando il movimento di entrambi per coprire l'intera area: ogni robot trascorre $3/4$ del tempo nella propria stanza esterna e $1/4$ in B, e i due non si trovano mai in B contemporaneamente. La stanza B resta quindi coperta per metà del tempo.

Si noti che in tutte e tre le strategie la somma delle inattività vale 1, quindi l'**inattività media è sempre** $1/3$: rispetto all'obiettivo formale le tre strategie sono equivalenti. Tuttavia la **terza strategia è la migliore**: nella pratica, per coprire il più possibile tutte le diverse stanze in modo omogeneo, è fondamentale introdurre il concetto di *equità*. Per garantire questa equità, limitarsi a valutare la media non è sufficiente: risulta necessario introdurre il calcolo della **varianza** o della **deviazione standard**, in modo da penalizzare gli squilibri estremi tra le stanze.

2.3 PEAS

Per progettare un agente intelligente bisogna definire l'ambiente in cui opera:

- **Performance Measures:** metodo di valutazione del successo dell'agente
- **Environment:** il contesto in cui l'agente opera
- **Actuators:** i mezzi attraverso cui l'agente agisce sull'ambiente
- **Sensors:** i mezzi attraverso cui l'agente percepisce l'ambiente

Esempio: Prendiamo in considerazione un taxi automatico: il PEAS potrebbe essere:

Performance measures	Soddisfazione del cliente, sicurezza, efficienza del carburante (SOLO E85), rispetto per le leggi stradali
Environment	Traffico stradale, condizioni meteorologiche, segnali stradali, pedoni e altri veicoli
Actuators	Volante, acceleratore, freni, indicatori di direzione
Sensors	Telecamere, GPS, sensori di velocità, radar

2.4 Tipi di Ambiente

Gli ambienti possono essere classificati secondo diverse caratteristiche:

- **Osservabile** (vs *parzialmente osservabile*): se l'agente può percepire completamente lo stato dell'ambiente in ogni momento
- **Deterministico** (vs *stocastico*): se lo stato successivo dell'ambiente è determinato completamente dallo stato corrente e dall'azione eseguita dall'agente
- **Episodico** (vs *sequenziale*): se l'esperienza dell'agente è divisa in episodi indipendenti, cioè l'azione in un episodio non influisce sugli episodi successivi
- **Statico** (vs *dinamico*): se l'ambiente non cambia mentre l'agente sta prendendo una decisione
- **Discreto** (vs *continuo*): se stati, percezioni, azioni e tempo assumono un numero finito di valori distinti
- **Singolo Agente** (vs *multi-agente*): se l'agente opera da solo nell'ambiente senza la presenza di altri agenti
- **Noto** (vs *ignoto*): se l'agente (o il progettista) conosce le regole dell'ambiente, cioè gli esiti delle azioni

Approfondimento – Scenario Singolo Agente in contesti Multi-Agenti

Un ambiente multi-agente può essere considerato singolo agente in quelle circostanze in cui si prevede la progettazione di un singolo agente mentre gli altri agenti non vengono modellati (che quindi vengono considerati elementi dell'ambiente).

Esempio: Tipi di ambienti in diversi contesti

	Crossword	Robo-selector	Poker	Taxi
Observable	Yes	Partly	Partly	Partly
Deterministic	Yes	No	No	No
Episodic	No	Yes	No	No
Static	Yes	No	Yes	No
Discrete	Yes	No	Yes	No
Single-agent	Yes	Yes	No	No

Durante il corso ci soffermeremo sugli ambienti *osservabili* e *deterministici* per descrivere problemi sequenziali: un'azione svolta ha un impatto sul futuro.

2.5 Tipi di Problemi

- **Deterministico, completamente osservabile $\Rightarrow$ problema a stato singolo (Single-State Problem).**
L'agente sa esattamente in quale stato si troverà; la soluzione è una sequenza di azioni.
- **Non osservabile $\Rightarrow$ problema conformante.**
L'agente potrebbe non avere idea di dove si trovi: l'agente non dispone di sensori. la soluzione (se esiste) è una sequenza di azioni.
- **Non deterministico e/o parzialmente osservabile $\Rightarrow$ problema di contingenza.**
Le percezioni forniscono costantemente nuove informazioni sullo stato corrente; la soluzione è un piano contingente (basato su condizioni del tipo if-then) o una *policy* (mapping stato-azione): molto spesso si alternano le fasi di ricerca e di esecuzione.
- **Spazio degli stati sconosciuto $\Rightarrow$ problema di esplorazione (*online*).**
L'agente deve scoprire gli stati e le transizioni interagendo direttamente con l'ambiente per capire come funziona il problema.

2.6 Programma Agente

Poiché l'agente è composto da un'architettura interna e un **programma agente**, è utile definire le caratteristiche generali di un programma agente.

In generale, lo scheletro di un programma è composto da:

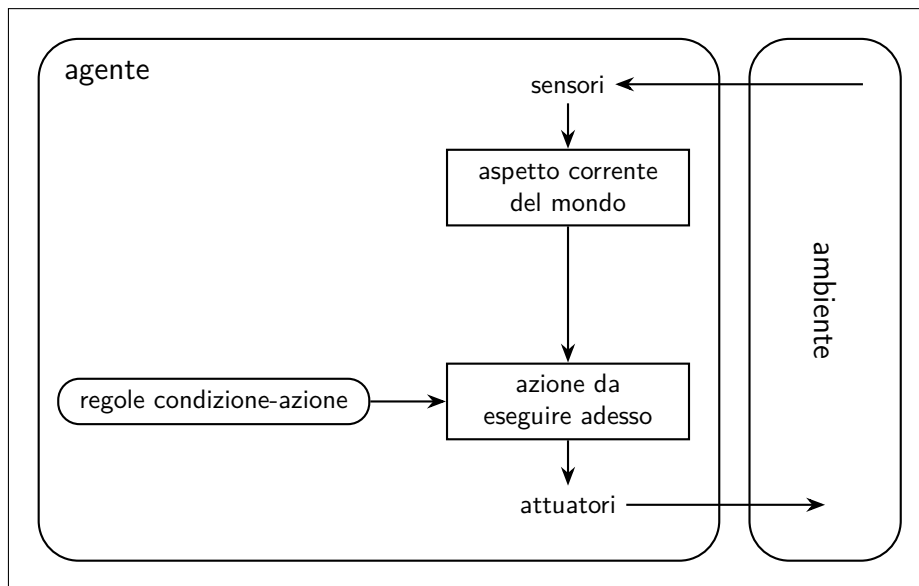
- Input: Percezione **corrente**. In tutti quei contesti in cui l'agente necessita di una sequenza di percezioni, ogni percezione dovrà essere salvata per riassemblare l'intera sequenza
- Output: **Prossima azione**

In questo schema è possibile seguire alcune architetture che consentono la modellazione di diversi agenti. Architetture diverse hanno diversi scopi. Di seguito l'elenco dei quattro tipi base di quasi tutti i sistemi intelligenti:

- Simple Reflex Agents
- Reflex Agents with State
- Agenti basati su obiettivi (Goal-Based Agents)
- Agenti basati sull'utilità (Utility-Based Agents)

2.6.1 Simple Reflex Agents

L'architettura più semplice è l'**Agente Reattivo Semplice**. Questi agenti scelgono le azioni sulla base della percezione corrente, ignorando tutta la storia percettiva precedente. Pertanto, il principio seguito è quello del tipo **regola condizione-azione**.



Simple Reflex Agents

```
function AGENTE-REATTIVO-SEMPLICE(percezione) returns azione
  persistent: regole          // insieme di regole condizione-azione

  stato ← INTERPRETA-INPUT(percezione)
  regola ← REGOLA-CORRISPONDENTE(stato, regole)
  azione ← regola.AZIONE
  return azione
```

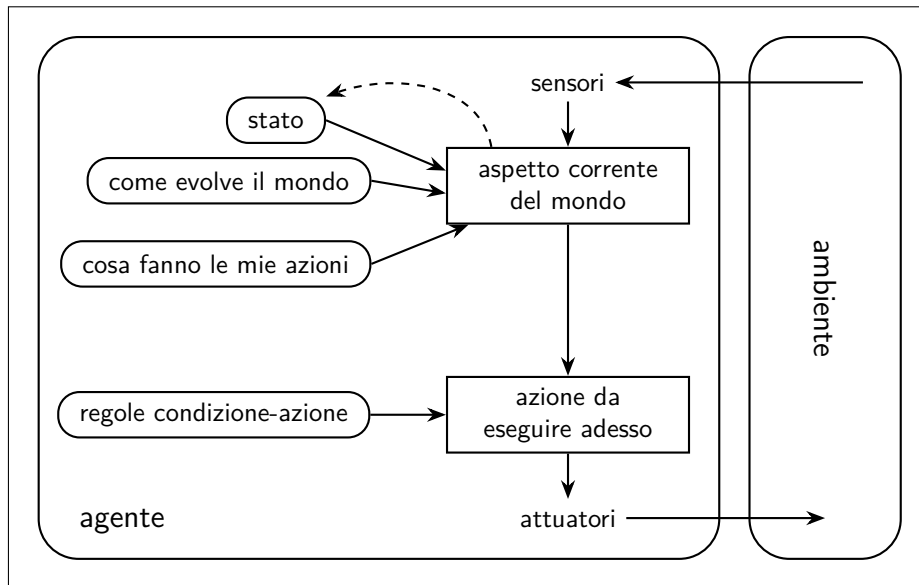
Gli agenti reattivi semplici hanno il pregio della semplicità, ma un'intelligenza molto limitata: funzionano correttamente **solo se l'ambiente è completamente osservabile**, cioè se la decisione giusta può essere presa sulla base della sola percezione corrente. Anche una minima parte di non osservabilità può causare seri problemi.

Esempio: se l'aspirapolvere non avesse il sensore di posizione e percepisse soltanto [Pulito] o [Sporco], davanti a una stanza pulita dovrebbe scegliere sempre la stessa azione, ad esempio *Sinistra*. Se si trova in A, quell'azione fallisce sempre: l'agente entra in un **ciclo infinito**.

2.6.2 Reflex Agents with State

Il modo più efficace di gestire l'osservabilità parziale, per un agente, è tener traccia della parte del mondo che non può vedere nell'istante corrente. Questo significa che l'agente deve mantenere una sorta di **stato interno** che dipende dalla storia delle percezioni e che quindi riflette almeno una parte degli aspetti non osservabili dello stato corrente. In particolare, aggiornare lo stato interno richiede due componenti fondamentali: gli effetti delle azioni dell'agente e le modalità di evoluzione del mondo indipendentemente dall'agente. Tale conoscenza viene implementata tramite un **modello di transizione**. Inoltre, sono necessari **modelli sensoriali** che descrivono come lo stato del mondo si rifletta nelle percezioni dell'agente.

Un agente che utilizza tali modelli prende il nome di **Agente Basato su Modello**.



Reflex Agents with State

```
function AGENTE-REATTIVO-BASATO-SU-MODELLO(percezione) returns azione
persistent:
    stato           // concezione corrente dello stato del mondo
    modello_transizione // stato successivo in funzione di stato e azione
    modello_sensoriale // come lo stato si riflette nelle percezioni
    regole          // insieme di regole condizione-azione
    azione          // l'azione più recente, inizialmente nessuna

    stato ← AGGIORNA-STATO(stato, azione, percezione,
                           modello_transizione, modello_sensoriale)
    regola ← REGOLA-CORRISPONDENTE(stato, regole)
    azione ← regola.AZIONE
    return azione
```

2.6.3 Goal-Based Agents

La conoscenza dello stato corrente dell'ambiente non è sempre sufficiente per decidere quale azione intraprendere. Spesso è necessario introdurre informazioni relative all'**obiettivo** (*goal*), ovvero la descrizione delle situazioni desiderabili da raggiungere. Un agente basato su obiettivi (*Goal-Based Agent*) combina queste informazioni con il proprio modello interno del mondo, superando il limite delle semplici regole "condizione-azione". Non basta più, infatti, associare direttamente una percezione a un'azione prestabilita. A differenza degli agenti reattivi (*Reflex Agents*), in cui l'obiettivo è solo implicito nelle regole pre-programmate, l'agente basato su obiettivi valuta in modo esplicito le conseguenze delle proprie scelte: prima di agire, analizza atti-

vamente se lo svolgimento di una determinata azione lo porterà o meno al soddisfacimento del proprio scopo.

Esempio: L'agente reattivo frena quando vede le luci di frenata, senza rendersi conto del motivo. Un agente basato su obiettivi frena quando vede le luci di frenata perché quella è l'unica azione che, secondo la sua previsione, permette di raggiungere l'obiettivo di non tamponare altre macchine.

Ricerca e pianificazione sono sottocampi dell'IA dedicati a identificare le sequenze di azioni da intraprendere per raggiungere gli obiettivi.

Può sembrare che un Goal-Based Agent sia meno efficiente (magari l'agente che modella le funzionalità di guida autonoma potrebbe frenare invece di accelerare su una strada dritta ad alta velocità), ma garantisce maggiore **flessibilità**, perché la conoscenza che guida le sue decisioni è rappresentata esplicitamente e può essere modificata.

2.6.4 Agenti Basati Sull'Utilità

Raggiungere semplicemente un obiettivo non basta: per un risultato ottimale è fondamentale valutare anche la qualità e l'efficienza delle azioni scelte per ottenerlo (come velocità, sicurezza o convenienza).

In questo ambito, infatti, si introduce il concetto di **Utilità**: all'interno dell'agente viene implementata una **Funzione di utilità** che internalizza la performance measure. Purché la funzione di utilità interna e la misura di prestazione esterna concordino, un agente che sceglie le azioni per massimizzare l'utilità sarà comunque razionale in base alla misura di prestazione esterna.

Si sottolinea che questo non è l'unico modo per essere razionali: l'esempio sull'aspirapolvere mostra la modellazione di un comportamento razionale senza introdurre obiettivi o funzioni di utilità. Ma anche in questo caso i vantaggi sono principalmente in termini di flessibilità, soprattutto in contesti in cui è necessario prendere decisioni in condizioni d'incertezza.

In termini tecnici, un agente razionale basato sull'utilità sceglie l'azione che massimizza l'**utilità attesa** dei risultati, ovvero l'utilità che l'agente si attende di ottenere, in media, date le probabilità e le utilità di ciascun risultato.

Esempio: Si immagini un taxi a guida autonoma che deve scegliere tra due strade:

- **STRADA A (Urbana):** è sempre libera (100%) e porta a destinazione nei tempi previsti, con utilità moderata pari a 5.
Utilità attesa: $1.0 \times 5 = 5$
- **STRADA B (Autostrada):** è libera nell'80% dei casi, offrendo la possibilità di arrivare in largo anticipo, con utilità elevata pari a 10. Nel restante 20% dei casi, invece, si trova un ingorgo totale e si arriva mol-

to in ritardo, con utilità = -5.

Utilità attesa: $(0.8 \times 10) + (0.2 \times -5) = 8 - 1 = 7$

La strada che massimizza l'utilità attesa è pertanto la B.

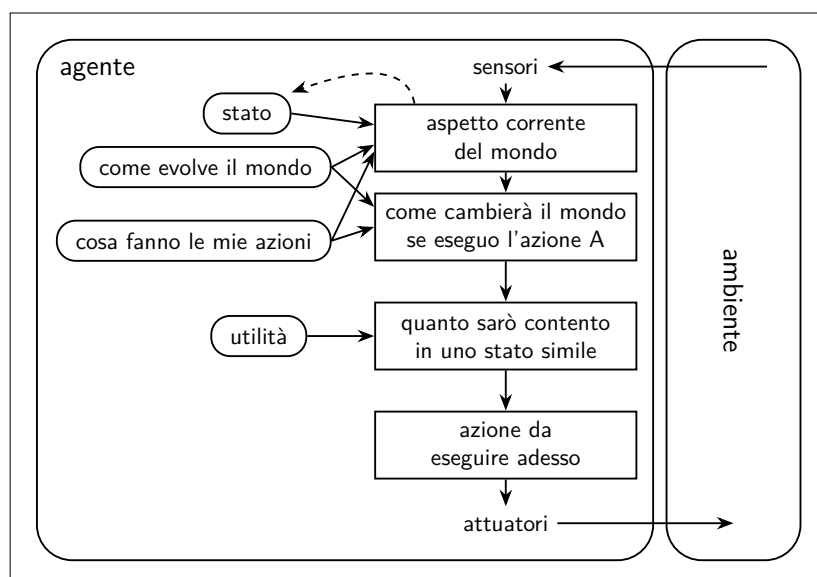
Approfondimento – Utilità Attesa

Si noti che non è altro che il calcolo del valore medio (*speranza matematica*): $E[X] = \sum x_i \cdot P(X = x_i)$

Differenza – Funzione di Utilità vs Performance Measure

Sebbene rappresentino concetti assai simili, la Performance Measure è il metro di giudizio **esterno**, stabilito dal progettista per valutare la qualità complessiva del comportamento dell'agente: l'agente non ha necessariamente accesso diretto a essa.

La funzione di utilità è invece **interna** all'agente: è l'internalizzazione della performance measure, costruita dal progettista in modo che le due siano allineate. L'agente la usa per assegnare un valore a ogni possibile stato del mondo e, in condizioni di incertezza, sceglie l'azione che massimizza l'utilità attesa. Se la funzione di utilità non rispecchia fedelmente la performance measure, l'agente può massimizzarla e comportarsi comunque in modo non razionale rispetto al criterio esterno.



Capitolo 3

Risolvere Problemi con la Ricerca

3.1 Problem Solving Agents

Quando l'azione da svolgere non è subito evidente, un agente potrebbe avere la necessità di guardare avanti, cioè considerare una sequenza di azioni (*cammino*) che porta a uno stato obiettivo. Questo tipo di agente viene chiamato **Agente Risolutore di Problemi** (*Problem Solving Agent*). Gli agenti risolutori di problemi utilizzano rappresentazioni **atomiche** degli stati, i quali vengono considerati come entità prive di una struttura interna.

I Problem Solving Agents sono in particolare una sottocategoria dei Goal-Based Agents. Per decidere quale azione intraprendere, questi agenti adottano un approccio sistematico sequenziale basato sulla formulazione di problemi, determinazione di obiettivi e sulla ricerca di soluzioni. Tale processo si divide in quattro macro-fasi:

1. **Formulazione dell'obiettivo (*Goal Formulation*)**: dato lo stato corrente dell'ambiente, l'agente definisce una situazione desiderabile (l'obiettivo) da raggiungere.
2. **Formulazione del problema (*Problem Formulation*)**: l'agente decide quali stati e quali azioni prendere in considerazione per raggiungere tale obiettivo.
3. **Ricerca (*Search*)**: l'agente esplora lo spazio degli stati simulando diverse sequenze di azioni nel proprio modello interno, fino a trovare una soluzione valida (una sequenza di mosse che porta all'obiettivo).
4. **Esecuzione (*Execution*)**: l'agente esegue, una alla volta, le azioni previste dalla soluzione trovata, ignorando le percezioni.

Notare come la risoluzione del problema in questo caso è di tipo **offline**: la soluzione viene eseguita a “occhi chiusi” (come detto prima, si ignorano le percezioni).

Differenza – Online vs Offline Problem Solving

A differenza della tecnica offline, la quale può essere usata in ambienti completamente deterministici e osservabili, la **Online Problem Solving** prevede che l'agente agisca in condizioni di parziale ignoranza, dove la conoscenza sull'ambiente non è completa. Pertanto è indispensabile che i sensori rimangano attivi per valutare l'esito delle proprie azioni.

Di seguito un'implementazione di un programma agente per un Problem Solving Agent che procede alla risoluzione di un problema mediante ricerca nello spazio degli stati.

Simple Problem-Solving Agent

```
function AGENTE-RISOLUTORE-SEMPLICE(percezione) returns azione
  persistent:
    seq          // sequenza di azioni, inizialmente vuota
    stato        // descrizione dello stato corrente del mondo
    obiettivo    // un obiettivo, inizialmente nullo
    problema     // la formulazione del problema

  stato ← AGGIORNA-STATO(stato, percezione)
  if seq è vuota then // la prima volta che si esegue
    obiettivo ← FORMULA-OBIETTIVO(stato) // che si vuole raggiungere
    problema ← FORMULA-PROBLEMA(stato, obiettivo)
    seq ← RICERCA(problema)
    if seq = failure then return null
  azione ← PRIMO(seq)
  seq ← RESTO(seq)
  return azione
```

Da notare come l'agente sceglie cosa fare **solamente** all'inizio e poi continua, ignorando completamente le percezioni.

3.2 Come Formulare un Problema

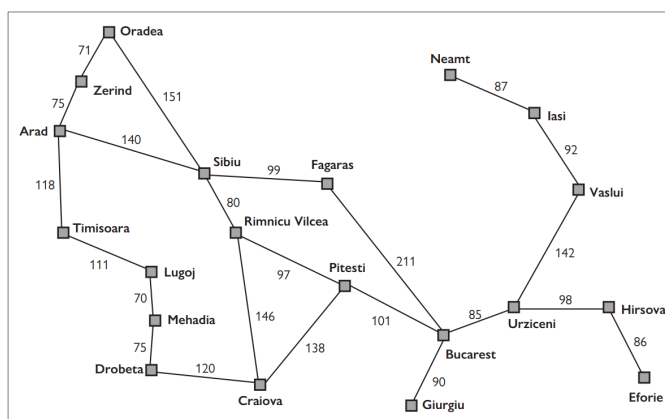
3.2.1 Problema: Vacanze in Romania (con un M5 e39)

Ci troviamo in Romania e dobbiamo andare a Bucarest per poi prendere l'aereo e tornare a casa. Attualmente ci troviamo ad Arad. Vogliamo trovare la

serie di città da attraversare per giungere a Bucarest. Dobbiamo individuare un corretto livello di astrazione, ma ragionevolmente si procede nel seguente modo:

- **Obiettivo:** andare a Bucarest
- **Formulazione del problema**
 1. **states:** le diverse città
 2. **actions:** guidare per attraversare le città
- **Soluzione da Individuare:** sequenza di città da attraversare per arrivare a Bucarest

Fin da subito è possibile mettere in discussione il livello di deterministicità del problema. A causa di agenti esterni (incidenti, lavori) si potrebbe essere dirottati su città non scelte. Tali supposizioni vengono inizialmente ignorate. In questo caso vogliamo trovare **una** soluzione: non è detto che quella trovata sia a costo minimo.



Cerchiamo di proporre una formulazione di tale quesito mediante Single-State problem. Allora si individuano i seguenti elementi:

1. **Stato Iniziale:** la città di partenza.
2. **Successor Function** che assume la seguente forma: $f : S \times A \rightarrow S$. Si mappa uno stato e un'azione in un altro stato. *Esempio:* $S(A) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots \}$
3. **Goal Test** che può essere di due nature diverse:
 - (a) esplicito: la città di destinazione (Bucarest). Lo stato in cui ci si deve trovare affinché il problema possa considerarsi risolto.
 - (b) implicito: una feature che descrive più di uno stato.

4. **Path Cost:** Non sempre si desidera trovare una soluzione qualunque. Potrebbe essere utile trovare quella a costo minimo. Nel nostro esempio corrisponde ad una metrica che indica il costo di un percorso (il costo espresso sugli archi). Il path cost è un processo additivo in cui vengono sommati gli **step cost**, funzioni del tipo $STEP_COST = S \times A \times S \rightarrow \mathbb{R}^+$.
5. **Soluzione:** sequenza di azioni che porta dallo stato iniziale a uno dei goal-state.

3.3 Tree-Search Algorithm

L'idea di base è fare un'esplorazione simulata e offline (essendo un single state problem) le percezioni vengono ignorate. Concettualmente l'agente ragiona, individua una sequenza di azioni e procede a eseguirla. Pertanto si generano dei successori degli stati usando regole che si conoscono del mondo.

Tree-Search Algorithm

```
function TREE-SEARCH(problema, strategia) returns una soluzione, o failure
  inizializza l'albero di ricerca usando lo stato iniziale del problema

  loop do
    if nessun candidato per l'espansione then return failure
    scelgo uno dei nodi per l'espansione seguendo la strategia
    if nodo contiene un goal-state then return soluzione
    else aggiungi il nodo successore al search tree (e espando)
  end
```

3.3.1 Stati vs Nodi

Nell'albero di ricerca è importante distinguere due concetti che a prima vista possono sembrare equivalenti:

- uno **stato** è la rappresentazione di una configurazione fisica del mondo: descrive *dove* si trova l'agente (ad esempio, "sono ad Arad");
- un **nodo** è una **struttura dati** che costituisce parte dell'albero di ricerca: oltre allo stato a cui corrisponde, memorizza *come* ci si è arrivati.

I campi di un nodo sono:

State	lo stato del mondo a cui corrisponde il nodo
Parent	il nodo da cui è stato generato
Action	l'azione applicata al padre per generare il nodo
Children	i nodi generati espandendo il nodo
Depth	il numero di passi dalla radice
Path cost $g(x)$	il costo del cammino dalla radice fino al nodo

Differenza – Stato vs Nodo

Gli stati **non** hanno padre, azioni, figli, profondità o costo di cammino: queste informazioni dipendono dal *percorso* seguito e non dal mondo. Per questo lo **stesso stato** può comparire in **più nodi diversi** dell'albero di ricerca.

Esempio: nel problema della Romania, lo stato “sono a Sibiu” può essere raggiunto:

- direttamente da Arad: profondità 1, $g = 140$;
- passando per Arad $\rightarrow$ Zerind $\rightarrow$ Oradea $\rightarrow$ Sibiu:
profondità 3, $g = 75 + 71 + 151 = 297$.

Si ottengono quindi due nodi distinti, con padre, profondità e costo diversi, che contengono però lo stesso stato. In sintesi: lo stato indica *dove* si trova l'agente, il nodo indica anche *come* ci è arrivato.

3.3.2 Un'implementazione di più basso livello

Lo pseudocodice sopra descritto risulta essere di carattere estremamente generale, pertanto si pone ad un livello di astrazione tale che risulta estremamente complesso procedere con una sua implementazione. Ecco perchè si procede generalmente nell'utilizzare un pseudocodice che si occupa di una descrizione di più basso livello

Tree-Search Algorithm

```
function TREE-SEARCH(problema, frontiera) returns una soluzione, o failure
  frontiera <- INSERISCI(CREA-NODO(problema.stato_iniziale))
  while not VUOTA(frontiera) do
    nodo <- POP(frontiera)
    if problema.GOAL_TEST(nodo.stato) then return nodo
    frontiera <- INSERISCI_TUTTI(ESPANDI(nodo, problema))
  end loop
  return failure
```

La frontiera non è altro una struttura dati che contiene nodi, disposti in un certo ordine. Il modo in cui i nodi vengono inseriti nella frontiera determina il tipo di strategia che si sta utilizzando (Breadth First Search, Depth First Search, Best First Search ...).

Notiamo come la strategia non è più un parametro in input, ma è intrinseca nel modo in cui i nodi vengono inseriti nella frontiera. Prima di tutto, inseriamo nella frontiera lo stato iniziale. Fino a quando la frontiera non è vuota itero estraendo il primo nodo dalla frontiera (inseriti a seconda della priorità definita nella strategia) e controllo se il nodo contiene il *goal state*. In caso affermativo, il nodo viene restituito, altrimenti si espande il nodo appena estratto dalla frontiera basandomi sulle regole presenti nel problema e poi si procede a inserire ciò che si è ottenuto con l'espansione all'interno della frontiera seguendo un determinato ordine. L'ordine stesso determina il modo in cui si effettua la ricerca.

Di seguito lo pseudocodice della funzione ESPANDI:

Expand Function

```
function ESPANDI(nodo, problema) returns un insieme di nodi
  successori <- INSIEME-VUOTO()
  for each azione, risultato in FUNZIONE-SUCCESSORI(problema, nodo.stato) do
    s <- CREA-NODO()
    s.padre <- nodo
    s.azione <- azione
    s.stato <- risultato
    s.path_cost <- nodo.path_cost +
                        STEP-COST(nodo.stato, azione, risultato)

    s.profondita <- nodo.profondita + 1
    AGGIUNGI(s, successori)
  end loop
  return successori
```

In particolare, analizziamo il suo comportamento:

1. La funzione FUNZIONE-SUCCESSORI restituisce tutte le azioni applicabili e i relativi stati risultanti dall'applicazione di tali situazioni. Se un'azione non altera l'ambiente, si ritorna nello stesso stato iniziale.
2. Il nuovo nodo non contiene solo lo stato di arrivo isolato, ma registra esplicitamente il nodo padre (attributo padre) e l'azione eseguita (attributo azione). Questa serie di puntatori è ciò che permette di ricostruire l'intero percorso (la soluzione) una volta raggiunto l'obiettivo.
3. Il calcolo $\text{node.Path-Cost} + \text{Step-Cost}$ accumula il costo effettivo del percorso dalla radice fino a quel punto esatto.

4. Raccolta dei risultati: Tutti i nuovi nodi istanziati e completamente popolati vengono aggiunti all'insieme successors, che viene poi restituito al chiamante.

3.3.3 Search Strategies

Una strategia è definita scegliendo l'**ordine di espansione dei nodi**. Le strategie sono valutate in base alle seguenti dimensioni:

- **completezza** : se c'è una soluzione, la ricerca è in grado di trovarla.
- **complessità temporale** : in funzione numero di nodi generati/espansi. si paga del tempo a generare e valutare un nodo
- **complessità spaziale** : numero massimo di nodi in memoria. Tutti i nodi che sono in memoria sono stati generati, ma potrei aver la necessità di tenere in memoria meno nodi rispetto a quelli generati.
- **ottimalità** : trova sempre la soluzione a costo minimo (quindi si desidera ottimizzare il path_cost, non la profondità)

Differenza – Generated vs Expanded

Qui il comportamento delle due procedure è molto diverso: la **generazione** procede a istanziare un nuovo nodo allocandone spazio e inizializzandone i suoi attributi. Analogo al concetto di costruttore nella OOP. Mentre, l'espansione prende un nodo e le regole di un problema per procedere a generare tutti i nodi figli da poi inserire nella frontiera.

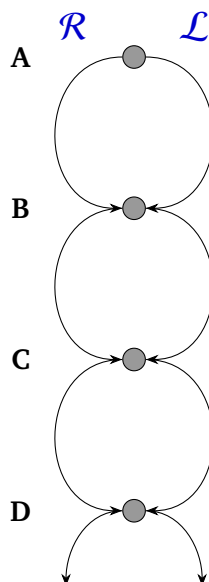
La complessità temporale e spaziale sono misurate in termini di:

- b : fattore di diramazione massimo dell'albero di ricerca. In parole meno tecniche, il massimo numero di figli di ciascun nodo.
- d : profondità della soluzione a costo minimo.
- m : profondità massima dello spazio degli stati (può essere ∞)

Notiamo come m è dato dallo spazio degli stati, mentre d dalla soluzione a costo minimo (tipicamente $d \ll m$).

3.3.4 Stati Ripetuti

Dato uno stato, esistono diverse azioni che mi consentono di raggiungere sempre lo stesso prossimo stato



Si osservi che lo stato B viene rappresentato nella frontiera **da 2 nodi distinti**: il primo generato dall'azione $\mathcal{R}$ e il secondo dall'azione $\mathcal{L}$. Questa ridondanza fa sì che la complessità del problema cresca notevolmente, assumendo un andamento di tipo esponenziale.

Sebbene in generale si preferisca evitare l'espansione di stati ripetuti, è fondamentale prestare attenzione poiché, in certi contesti, mantenere l'esplorazione di questi percorsi multipli è cruciale per garantire l'ottimalità. Infatti, se un nodo appena espanso corrisponde allo stato obiettivo (goal) e la ricerca viene interrotta immediatamente, si rischia di accettare una soluzione non ottimale.

Facendo riferimento alla figura, supponiamo che B sia lo stato obiettivo e che il costo del passo (step cost) associato all'azione $\mathcal{R}$ sia maggiore rispetto a quello di $\mathcal{L}$. Se l'algoritmo si fermasse all'istante subito dopo aver raggiunto B tramite $\mathcal{R}$, la soluzione a costo minimo (ovvero quella ottenibile tramite l'azione $\mathcal{L}$) verrebbe ignorata e mai individuata.